arXiv:2606.02142v1 [cs.LG] 1 Jun 2026

TimeBlocks: Foundational and Continual Time-Series Blockbase—Extended Version

David Campos
Aalborg University, Denmark
University of Amsterdam
Netherlands
dgcc@cs.aau.dk

Bin Yang*
East China Normal University
China
byang@dase.ecnu.edu.cn

Tung Kieu*
Aalborg University
Denmark
tungkvt@cs.aau.dk

Lei Chen
Hong Kong University of Science and Technology, Guangzhou, China
leichen@cse.ust.hk

Chenjuan Guo
East China Normal University
China
cjguo@dase.ecnu.edu.cn

Christian S. Jensen
Aalborg University
Denmark
csj@cs.aau.dk

Abstract

The ongoing digitization has led to a proliferation of time-series data streams that monitor a variety of processes, from which valuable insights may be obtained. Further, the emergence of successful foundational language models begs the question of whether it is possible to achieve time-series models with the foundational properties of handling multiple tasks, while being sufficiently lightweight to allow real-time data stream processing. Existing foundational time-series models are often large and only effective in offline settings without stringent time and computational constraints, and where repeated model calibration is not needed. However, when applied to data streams, these models are ineffective due to their size and lack of support for continual calibration, which compromise their ability to deliver accurate real-time responses, their durability, and their deployability in hardware-limited settings. We propose `TimeBlocks` to enable versatile time-series processing by facilitating the efficient building of lightweight models suitable for multiple tasks under variable conditions. In particular, the method maintains a pool of interchangeable and modular model blocks that can be used to construct new time-series models. When presented with specific time-series data, a routing strategy iteratively selects the most suitable blocks to construct a lightweight and accurate model for the data. We equip `TimeBlocks` with a method called `StreamCore` to build a representative small subset of the data stream, which preserves a guaranteed approximation of the stream over time, enabling continual model calibration. An experimental study on multiple data sets and covering multiple tasks shows that `TimeBlocks` enables to build models capable of outperforming existing baselines.

1 Introduction

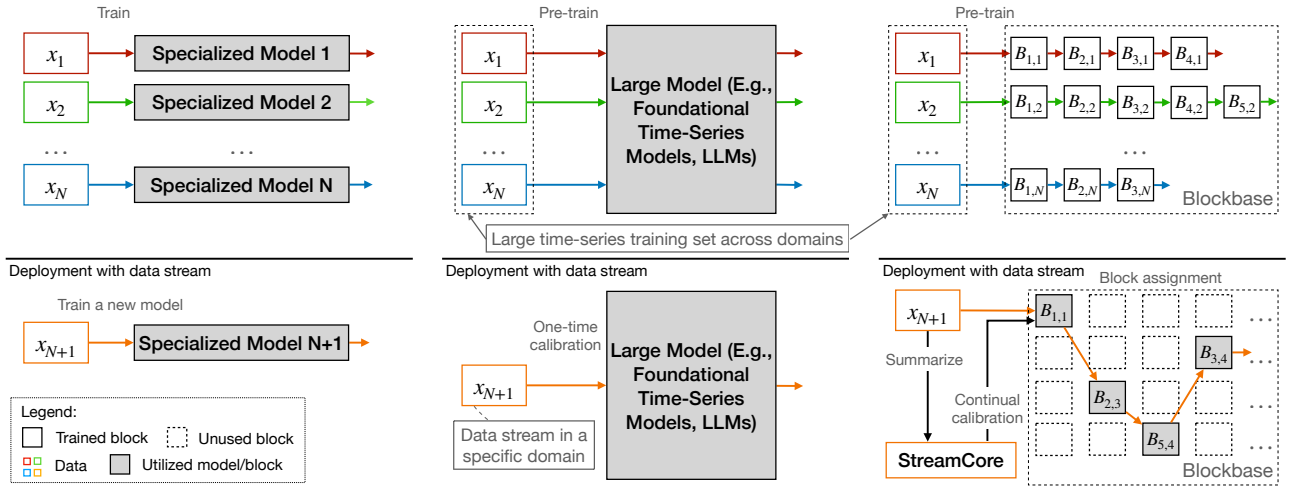
Due to the ongoing digitalization of societal and industrial processes, we are witnessing an increasing availability of data streams that have the potential to offer valuable insights into processes and to enable new or improved applications. With the emergence of powerful and foundational large language models—i.e., models trained across multiple data sets and settings, thus having good generality—there is a growing interest in exploiting the impressive capabilities of such models in applications beyond language processing, such as analyzing data streams. To enable value creation from data streams, methods must possess excellent analytical capabilities while also being lightweight and enabling real-time responses. Such methods hold the potential to enable applications in settings where decisions need to be made fast and where computational resources are limited. For example, in intelligent vehicles, in-vehicle controllers perform real-time tasks, such as forecasting power consumption under different driving conditions [40, 71] or detecting possible anomalies in the sensors [25, 31]. Due to hardware limitations in this type of environment, methods that are small in size and can handle multiple settings and tasks are called for.

Therefore, the problem is to develop models that can handle multiple tasks and data sets, while also being able to operate and be updated in computationally constrained environments, such as those of edge devices. This is an attractive way to handle the same type of data, i.e., time series, without deploying separate models for each task. Additionally, it enhances the capabilities of edge devices and maximizes their resource utilization. Recent studies attempt to apply foundational large language models to time series directly as these and text share sequential characteristics [52, 77]. However, this approach is usually inefficient, given the large sizes of language models [66, 79], and it does not generalize sufficiently, as time series embody specific characteristics that benefit from more specific models [51, 74].

Although a few studies aim to address some of these specific requirements of time series [34, 52, 64], limitations remain that need to be addressed. In particular, existing foundational time-series models remain very large [14, 73, 75, 79]. Then, given their sizes and data requirements [37, 75], these models are not suitable in a resource-limited environment and in settings that require continual calibration or fine-tuning. This is necessary to maintain performance over time, for example, on embedded sensor controllers. We proceed to describe two key limitations of current state-of-the-art methods for developing foundational time-series models, after which we give an overview of how to address them.

Large Pre-Trained Models: Traditionally, when processing multiple tasks and data sets, a specialized model is trained for each case, as shown in Figure 1(a). This approach is inefficient as it entails the creation of a substantial number of independent models. Current foundational time-series models have been developed to address this issue by allowing for the handling of multiple tasks

*Corresponding authors.



(a) Specialized Models Paradigm. **(b) Large Model Paradigm.** **(c) The `TimeBlocks` Paradigm.**

Figure 1: Paradigms for Time-Series Processing. (a) In the specialized model paradigm, a new model must be trained for each time series. (b) In the large model paradigm, a large model is pre-trained using a large set of time series across domains $\{x_i\}_{i=1}^{N}$. When deployed, model calibration requires sufficient amounts of data for a single calibration step. (c) In `TimeBlocks`, multiple small time-series processing blocks are pre-trained and stored in the `Blockbase`, and a data stream is summarized into a representative subset called `StreamCore`, allowing for continual calibration.

and data sets. However, this results in larger models, as depicted in Figure 1(b). This limitation suggests the need for a middle-ground solution between small, highly specific time-series models and large, more versatile foundational models, i.e., efficient small models that can handle heterogeneous time series among multiple tasks and analytic requirements.

Ineffective Continual Calibration: Given the large sizes of current foundational time-series models, calibration after deployment is usually conducted a single time due computational constraints, as shown in Figure 1(b). This approach is suitable mostly in offline settings with sufficient computational resources to accumulate all the necessary data and perform fine-tuning or calibration. It is not suitable in settings where the data characteristics may change over time, thus requiring continual calibration. Therefore, the continual calibration of large foundational models remains as an unresolved issue, as it is impractical to rapidly calibrate large models while accounting for the accumulation of sufficient data from the stream.

To address the limitations mentioned above, we propose `TimeBlocks` as a novel paradigm for continual deployment of efficient models for time-series analytics.

Addressing Challenge 1 (Small Model): Instead of using a large pre-trained model to analyze all time series, we propose a new paradigm that allows for the creation of lightweight and specialized models for time-series processing, as shown in Figure 1(c). Rather than training a single large model, our approach involves pre-training models constituted by blocks when considering a high variety of time-series tasks, data sets, and settings, such as historical windows, or forecasting horizons. The notion of a block is at the core of this paradigm. A block is a tiny and independent time-series processor, that when composed with other blocks can form models to perform different types of analysis. Thus, when designing and training models built upon blocks for time-series analytic tasks, it is possible to store the blocks of each model in a pool, that we called the `Blockbase`. Once in the `Blockbase`, a subset of these modular time-series blocks can be connected as needed to perform different analyses on time series with different characteristics, making the approach versatile for task processing.

The proposed paradigm dynamically selects the necessary blocks at inference time when analyzing a new time series, creating a specialized model that uses only the selected blocks to analyze it. This modular approach selects the pre-trained blocks based on the performance of the model under construction, aiming to improve it with each additional block. This is different from other strategies where combining several models imply more training, such as boosting ensembles [50] or recursive learned indexes [32].

Addressing Challenge 2 (Small Data Subset): To enable efficient continual calibration of a lightweight model, we propose a method, called `StreamCore`. This method reduces the computational overhead of accumulating a data stream by introducing a summarization strategy. The strategy maintains a subset with minimal cost updates that align with how often the model is calibrated. Thus, `StreamCore` enables the calibration of the built model over time, while also maintaining a low computational cost to fit within constrained hardware environments. Furthermore, the subset ensures a guaranteed representative approximation of the data stream while maintaining a low update cost.

In summary, the paper makes the following contributions:

- It proposes `TimeBlocks`, a novel paradigm where independent, modular, and stackable blocks allow for the building of time-series processing models, providing a high degree of flexibility to accommodate different time series and time-series tasks.
- It proposes `StreamCore`, a low-cost summarization method with approximation guarantees for the continual calibration of lightweight models under time-series data streams.

- It reports on experiments that provide insights into the effectiveness of `TimeBlocks` for constructing models and of `StreamCore` for calibrating them under data streams.

2 Preliminaries

We introduce key concepts required to present the paper proposals.

2.1 Time-Series Data Stream

2.1.1 Time Series. A time series $x \in \mathbb{R}^{M \times V}$ is a time-ordered and regularly sampled sequence of M timestamps, each associated with V variables. When $V = 1$, the time series is univariate, while when $V > 1$, it is multivariate.

2.1.2 Data Stream. A data stream is a time series x collected continuously over time, so the number of timestamps M is unbounded.

2.1.3 Analytics Function and Time Series Tasks. Let $f(\cdot)$ be a time-series analytics function that maps a time series from $\mathbb{R}^{C \times V}$ to $\mathbb{R}^{F \times V}$, when processing a time-series task. For instance, in forecasting tasks, $f(\cdot)$ uses variables from C historical timestamps to predict variables for the next F timestamps, such as predicting future electricity consumption. In imputation tasks, $f(\cdot)$ maps variables from C timestamps to the same space, so F is equal to C. Imputation can be used, for instance, for reconstructing missing sensor data in cases of communication errors and for outlier detection tasks, as it is unlikely that abnormal data can be reconstructed. In classification, the time series is mapped from $\mathbb{R}^{C \times V}$ to a set $\mathbb{L}$.

2.1.4 Context Length. The context length is the sequence of C timestamps in the domain of the analytics function $f(\cdot)$. For instance, in forecasting, the context length corresponds to the number of historical timestamps.

2.2 Time-Series Processing Blocks

Consider an analytics function $f(\cdot;\Theta)$ with parameters Θ that applies to a time series x_i. The parameters Θ may be disentangled as a sequence of P processing blocks, $\Theta := B_P(B_{P-1}(B_{\ldots}(B_2(B_1(x_i)))))$. Blocks may employ different architectures, including the Multilayer Perceptron (MLP), Long Short-term Memory Networks (LSTM), and attention mechanism. The residual output r_p from each block B_p is the input to block B_{p+1}, as shown in Figure 2.

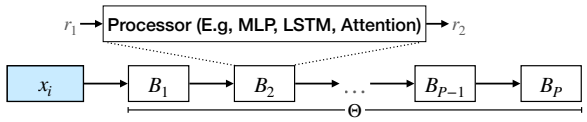


Figure 2: Time-Series Processing Blocks.

3 TimeBlocks

3.1 Problem Setting

We consider the problem of developing time-series processing models that can handle multiple tasks on the edge, while supporting calibration in a continual learning setting. An example is building specialized models that enable different types of forecasting in the multiple controllers found in modern vehicles. Typically, this task is accomplished by either training new models from scratch or by fine-tuning large existing models, both of which can be time-consuming given the many models needed. Section 3.2 presents the `Blockbase` routing framework that addresses these limitations. This framework enables the efficient construction of models using independent blocks that have been pre-trained. The method works at inference time, meaning that it builds a model while provides real-time results.

Then, when a model is deployed in a continual learning setting, its operating environment may change over time due to the nature of data streams. Therefore, to ensure the performance of such models over time, a common technique is to perform periodic calibration with new data. However, this approach can be computationally expensive as it entails retraining of large models, despite their generality, and the accumulation of all incoming stream data. To reduce computational costs, Section 3.3 presents means of building and maintaining a representative subset of a data stream that guarantees data diversity over time and enables periodic calibrations.

An overview of the proposal is shown in Figure 3. First, during a pre-training stage, we generate a pool of time-series processing blocks, named `Blockbase`, by training models constituted by blocks among multiple settings. Second, when presented with a new data stream, a model is constructed by selecting and stacking the most suitable processing blocks from the `Blockbase`. Third, to enable an efficient continual calibration, a compact representation of the data stream is maintained, so called `StreamCore`, ensuring that the most diverse data points are kept to facilitate continual calibration of the constructed model.

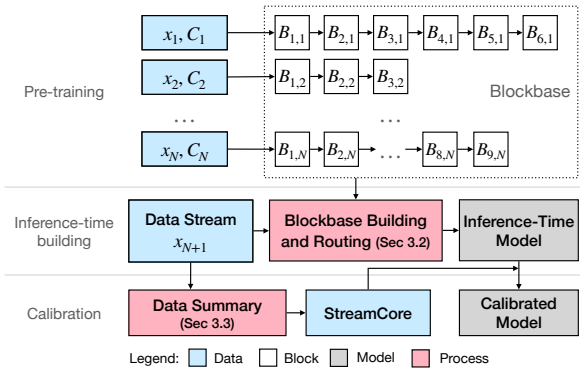


Figure 3: `TimeBlocks` Paradigm Overview.

3.2 Blockbase Routing

3.2.1 Overview. While numerous time-series models may be available for supporting multiple tasks, as shown in Figure 1(a), the fact that they concern the same type of data suggests that their processing is similar. Existing large models, as shown in Figure 1(b), leverage this similarity by training a one-fits-all large model to handle a wide range of time series. However, their large size limits their use in hardware-constrained settings. To address this, the proposed `Blockbase` decomposes this type of structure into independent processing blocks, as shown in Figure 1(c). Thus, when a new time

series needs to be analyzed, a small model can be built selecting the most relevant blocks. The process for building and utilizing the Blockbase is divided into three main components, which are detailed below.

3.2.2 Blocks Pre-Training. To generate blocks, we design end-to-end models using time-series processing blocks and train them using various tasks, data sets and training parameters. The complete details are available in the Appendix A.1. Then, the trained blocks from each model are included into the Blockbase for future use in building inference-time models. An end-to-end model is, as shown in Figure 2, a sequence of independent blocks. Each block is enhanced with a patching structure [47] that effectively handles specific time-series conditions, such as trends and cycles [16, 17], when processing the series in smaller segments. Also, we make the blocks modular to allow for their independent use at any position on new models when considering the following properties.

Multi-Scale Blocks: As time series exhibit high heterogeneity among frequencies [9, 48], it is important to process them while considering this variability, enabling to identify patterns at different scales. Therefore, for a given end-to-end model, we design the blocks to process different levels of granularity depending on their sequential position within the model. Specifically, we scale the input of each block, as shown in Figure 4 at the bottom, while following a decreasing power-of-two pattern [9, 48] to capture patterns from broad to specific. Thus, each level limits its input to double the previous level, as shown in Figure 4, at the top.

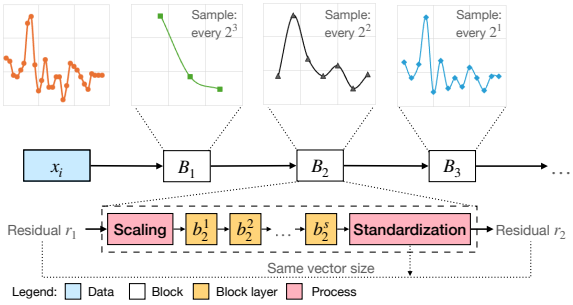


Figure 4: Multi-Scale Block Processing.

In addition to enable a better processing of a time series, this multi-scale strategy ensures that the blocks included in the Blockbase are capable of managing time series with multiple scales.

Output Standardization: In order to achieve independence and interchangeability among every block, it is important to establish a common interface that allows the interaction and connection between any pair of blocks. Thus, each block B_p incorporates a standardization layer for the output r_p, as it is shown in the below part of Figure 4. The operation involves a fully-connected layer that standardizes the output to a common vector size, regardless its scale or data set, enabling its compatibility with all types of blocks.

Diverse Settings: To achieve a variety of blocks, we train end-to-end models using a wide range of data sets, tasks, architectures, training parameters, and number of blocks. For example, on forecasting tasks, we use multiple context lengths and forecasting horizons [69]. As a result, Blockbase is highly diverse, the goal being to enable the processing of many types of time series when building new models.

Once all the blocks are trained, they are logically organized in the Blockbase, as described in Section 3.2.3. Then, the router enables their selection, as will be explained in Section 3.2.4.

3.2.3 Blocks Clustering. As the goal of the Blockbase is to have a large pool of time-series block processors, it is important to design a strategy to logically organize them efficiently in order to ensure a speedy block retrieval process.

Consider a block $B_p = \langle b_p^1, b_p^2, \ldots, b_p^s \rangle$ with s layers (e.g., a MLP with s linear layers). After training the block, the weights of all its layers remain constant, allowing these weights to serve as a means of distinguishing one block from another. In particular, we designate the first layer, b_p^1, as the primary identifier for the block, since this is the layer where the block first receives data flow from a previous block. We referred to it as the *fingerprint* because it serves as a distinctive identifier for the block training. Thus, the *fingerprint* $b_p^1 \in \mathbb{R}^{In \times V \times Out}$ is a homogeneous weight vector across all blocks, where *In* and *Out* represent the input and output sizes of the layer, while V refers to the number of variables.

To organize logically the blocks after their training, we cluster all the generated blocks using their fingerprint as the metric of comparison. Clustering provides flexibility as it allows for some fingerprints to be assigned to the most suitable cluster instead of directly assigning a group based solely on their parameters. The clustering assignment is done using the K-Means algorithm [29] with K equal to the number of considered context lengths, as experiments show that blocks trained with the same context length sometimes, but not always, share similar fingerprints. K-Means is applied to the complete set of blocks, and employs the block that is closest to the cluster center for further fingerprint comparisons.

3.2.4 Router. To build a new model using blocks in the Blockbase, we propose a routing mechanism that, when given a time series, iteratively builds a model by identifying the blocks that are most beneficial for processing it.

An overview of the router functionality is shown in Figure 5. Initially, when presented with a time series, the router identifies the most appropriate cluster of blocks. From there, it can select the most suitable block within the cluster, thereby also determining the context length needed for analyzing the time series. This is determined by the first block as it is the only one that processes the input time series directly. Then, the router selects each block based on the result of evaluating the model up to the last added block. For example, it selects the second block when evaluating the model using only one block. The selected blocks may include blocks with different scales, but there is no restriction on following a pattern. This process repeats until the required number of blocks is reached.

To design the routing mechanism, we use model features to assess the model status during the construction process. With the model parameters denoted as $\Theta := B_P(B_{P-1}(B_{...}))$, the residual r_p of each block B_p transitions to block B_{p+1}, as shown in Figure 2. This residual serves as a means to assess the model until block B_p. To map the relationship between the block residual r_p and the following block B_{p+1}, we employ the block fingerprint b_{p+1}^1 (see Section 3.2.3) as an identifier of the block, since it represents the first part of the block that interacts with its input. Therefore, we define a router function that establishes the link between two consecutive blocks as

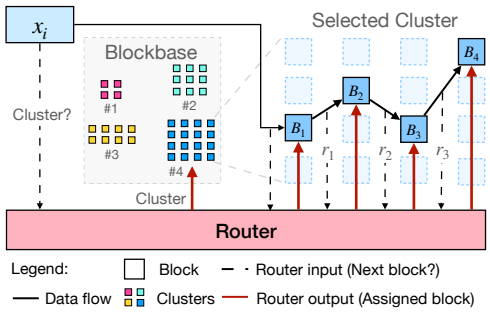


Figure 5: Router Overview.

$Router(r_p) \approx b^1_{p+1}$. This router aims to approximate the fingerprint of a block that matches the output r_p. Furthermore, each block is designed to produce a standardized output (see Section 3.2.2), ensuring that the mapping remains consistent across all residuals.

Block Selection Problem: The router is a heuristic proposed to efficiently identify a set of blocks, a process that is otherwise computationally expensive. Thus, the problem of selecting blocks for an inference-time model can be viewed as choosing a path of length j within the `Blockbase` that yields optimal performance. This problem can be considered a derivation of the Hamiltonian path problem, as detailed below.

Consider a set of blocks $\mathcal{B} = \{B_1, B_2, \ldots, B_N\}$ as the vertices of a graph. A path between two blocks $\mathcal{B}' \subset \mathcal{B}$ has a length given by the number of edges between them, and a cost given by the weight defined for those edges. For a given time-series task, it is reasonable to assume that a Hamiltonian path can be built, since the modular design of the blocks allows the N vertices to be connected by a path of length $N-1$. Finding such a path is an NP-hard problem, while, if such a path exists, there is one with minimal cost. Thus, the problem can be reduced to the case $j = N-1$, which is defined by the number of blocks requested to build a model.

To evaluate the cost of a path $c(\cdot)$, the simplest approach is to measure the performance gain from adding each block. However, this is prohibitively expensive, as it requires evaluating all blocks for every addition. Therefore, we must use a different type of metric to select the path.

To address this problem, alongside approximating an optimal solution for the Hamiltonian path, we introduce a router as a heuristic to enable the selection of blocks based on how the models are structured when trained independently. Thus, the metric that defines the cost between two blocks is implicitly learned when training the router. This training minimizes a loss between interconnected blocks, enabling the router to identify blocks that are similar to those that have processed analogous data in the independent models.

Considering that the router can approximate a Minimum Spanning Tree, as it is possible connects all vertices while minimizing total cost, and that every vertex can be associated with an edge (i.e., the graph has a minimal perfect matching), it has been shown under these assumptions [27] that the approximation for a Hamiltonian path $\mathcal{B}'_{ij}$ between two vertices B_i and B_j has a tight bound of 5/3 relative to the optimal solution $\mathcal{B}^*_{ij}$. Therefore, the cost of the path found by the router is bounded as follows:

$$c(\mathcal{B}'_{ij}) \leq c(Router(\mathcal{B}'_{ij})) \leq \frac{5}{3} c(\mathcal{B}^*_{ij})$$

Router Training: To obtain a $Router(\cdot)$ that can link consecutive blocks, we train an auxiliary model to learn to do so during the block pre-training stage. At this stage, the blocks of each model are optimally interconnected, and the data flows between the blocks is similar to what would be required when building a model in inference time. The architecture of the router is kept simple, consisting of two 1D convolutional layers followed by a fully-connected layer, as it is important to be able to select blocks efficiently.

The process for training $Router(\cdot)$ is shown in Figure 6 to the left. In this example, the router is trained to approximate the fingerprint b^1_2 of the next block B_2 using the residual r_1 from block B_1, such that Router$(r_1) \approx b^1_2$. The router is then trained across all blocks of the model and other trained models. The completed details are available in the Appendix A.2.

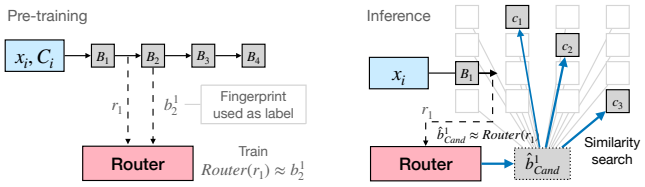


Figure 6: Router Training and Block Selection.

3.2.5 Inference-Time Model Building. To efficiently build an inference-time model, we use the `Blockbase` in conjunction with the router to select the blocks that are most suitable for processing a given time series. The router first selects the optimal cluster for processing the time series, thereby reducing the number of blocks to be considered for model construction.

Context Length Selection: Noticing that when selecting a block, we are also choosing its context length, we use this property to evaluate different context lengths when searching for a block and then choose the most suitable one. Further, as we organize the blocks into clusters to enhance search efficiency, as described in Section 3.2.3, the choice of context length is determined by evaluating potential clusters. This is because, in general, we experimentally observe that blocks within a cluster tend to share the same context length. Thus, as Figure 5 shows, when given a time series, the router, using the `Blockbase`, identifies the closest cluster while also determining the context length. This reduces the number of evaluations needed to select new blocks, another router task to be detailed shortly, as the selection of blocks is done limited to the cluster. Overall, the comparison uses the minimum distance as the metric for selecting blocks, as it indicates how closely the fingerprints are aligned and is fast to compute. More complex metrics could also be considered, provided they remain simple and effective.

Block Selection: After selecting the cluster from the `Blockbase`, the router builds the model by identifying the most suitable blocks within it. Considering a given time series x, the objective is to identify a suitable subset of blocks that allows to process x such as the analytics function (e.g., forecasting) has minimal error, while staying within a predetermined budget of J blocks. Formally, when considering the `Blockbase` as a set $\mathcal{B}$, the goal is to identify a set $\mathcal{B}' \subset \mathcal{B}$, $\mathcal{B}' = \{B_1, B_2, \ldots, B_J\}$, for the time series x and the analytics function $f(\cdot)$ so that when evaluating $f(\cdot)$ using $\mathcal{B}'$ yields the lowest possible error, while using at most J blocks, i.e., $|\mathcal{B}'| \leq J$.

To process a time series, the blocks in the set $\mathcal{B}'$ are connected sequentially. Each block takes an input, processes it, and directs the output residual to the next block for further processing. Therefore, the objective is to minimize a criterion $g(\cdot,\cdot)$ that aligns subsequent pairs of blocks to form a directed path $B_1 \rightarrow B_2 \rightarrow \cdots \rightarrow B_J$, which can be formulated as follows.

$$\arg\min_{\mathcal{B}' \subset \mathcal{B}} \sum_{j=1}^{\mathcal{B}} g(B_j, B_{j+1}) \quad s.t.\ |\mathcal{B}'| < J$$

To select subset $\mathcal{B}'$, the router is used in an iterative process, as illustrated in Figure 6 to the right, until reaching J blocks. Each time a block B_j is selected, the model is evaluated up to that block, and the resulting residual r_j is sent to the router to enable it to select the next block B_{j+1}. In Figure 6 to the right, the output r_1 from block B_1 is used by the router to calculate the candidate fingerprint for the second block $\hat{b}^1_{Cand}$. Then, through similarity search, the router compares this fingerprint with those in the cluster and selects the blocks with the most similar fingerprints, denoted as c_1, c_2, c_3, which can be performed with a cost on the order of $J \times \log(|\mathcal{B}|)$ [30]. The complete details are available in the Appendix A.3.

As the search process identifies a group of candidate blocks, multiple independent paths can be evaluated simultaneously, while ensuring that the results are not dependent solely on a single subset $\mathcal{B}'$. Thus, when assessing multiple tasks like forecasting and outlier detection, we can determine the most appropriate subset for each specific task, instead of applying a single subset to all tasks.

The selection of blocks is accelerated using vector processing techniques [11] and it is conducted at inference time. This results in a model tailored to the data stream in which it is used. We also enable periodic model calibration to accommodate any shifts in the data stream, as time series data can change due to variable environments or seasonality. This process is explained next.

3.3 StreamCore

When considering how to calibrate a model with a data stream, one important issue is how to effectively accumulate the data. Simple solutions, such as logging all the data, can be costly in terms of storage and computation, while basic buffer may not accurately represent the stream. Therefore, we consider computing a subset that can approximate the data stream while maintaining a small size to effectively calibrate the model and limiting the possible information loss. To achieve this, we first need to prove that building such a subset is feasible and efficient, as detailed as follows.

Consider an analytics function $f(\cdot)$ and a data stream $\mathcal{D}$, which are evaluated by a loss function $l(\cdot,\cdot)$. A subset $\mathcal{S} \subset \mathcal{D}$ is represented by a non-negative vector $w = [w_1, \ldots, w_n]$, where $w_i > 0$ indicates that a time series $x_i \in \mathcal{S}$. Then, a subset $\mathcal{S}$ of size m is considered a coreset of $\mathcal{D}$ when

$$w^* \in \arg\min_{w, \|w\|_0 = m} L(\mathcal{D}, w)$$

$$L(\mathcal{D}, w) = \frac{1}{n}\left\| \sum_{i \in \mathcal{D}}^{n} l(x_i, f(x_i)) - \sum_{j \in \mathcal{S}}^{n} w_j l(x_j, f(x_j)) \right\| \tag{1}$$

achieves highly similar performance with respect to evaluating the data set $\mathcal{D}$. Then, considering a streaming setting, a new data set $\mathcal{D}^t$ arrives at time t, so the goal becomes finding a subset $\mathcal{S}$ such that Equation 1 becomes

$$w^* \in \arg\min_{w^t, \|w^t\|_0 = m} L(\mathcal{D}^t, w^t), \tag{2}$$

where the subset $\mathcal{S}$ is updated at every t to maintain examples from the complete feature space.

Solving the coreset selection is usually an intractable problem since it is necessary to evaluate the function for all the examples [41, 57]. Also, for streaming settings, the computation should be efficient, so it becomes necessary to provide guaranteed approximations.

By considering Lipschitz continuity, Equation 1 can be bounded as follows (the derivation is provided in the Appendix A.4):

$$\frac{1}{n}\left\| \sum_{i \in \mathcal{D}}^{n} l(x_i, f(x_i)) - \sum_{j \in \mathcal{S}}^{n} w_j l(x_j, f(x_j)) \right\| \leq \frac{1}{n} \sum_{i \in \mathcal{D}}^{n} \min_{j \in \mathcal{S}} \lambda \|x_i - x_j\|$$

Thus, an approximate solution for the coreset can be found depending exclusively on the data without computing the function f for all the examples, i.e., the boundary. Therefore, it is only necessary to calculate the distance in the feature space of the examples to identify a subset that minimize the overall distances. This corresponds to the facility location problem, an NP-hard problem [43], so the goal is to provide solutions with approximation guarantees.

There are some solutions to this problem [35, 45] that provide guarantees to the offline method. They are typically α-approximate algorithms [44], meaning that their results are within a factor α from the optimal solution. For instance, $\alpha = 2$ algorithms exist with a running time of $\Phi = O(|\mathcal{S}| \times |\mathcal{D}^t|)$.

A simple extension of these algorithms to the streaming setting [35] involves computing the coreset and merging it every time t. The typical cost of each of these operations is $O(|\mathcal{S}| \times |\mathcal{D}^t|)$ over T periods, resulting in a total running time of $O((|\mathcal{S}| \times |\mathcal{D}^t|)^2 \times T)$. As this is computationally expensive, we introduce `StreamCore` as a method to reduce the cost of building subsets. It achieves this by avoiding the merging operation when updating a coreset.

Considering an existing α-approximate solution for the facility location, it has been demonstrated that it is possible to maintain a $(8\alpha + 4)$-approximation for the same problem by at most $R = \frac{\Phi}{4\alpha}$ arbitrary updates with at most a linear cost [13]. Thus, consider a simple algorithm that adds every new point with cost $O(1)$ until the size of the coreset is met. Once the size has been reached, for each new point added, an existing point should be removed. Randomly selecting an existing point in $\mathcal{S}$ and deleting the point that is closer has a cost of at most $|\mathcal{S}|$ while maintains the coreset diverse, as removals are uniformly distributed.

Then, as each time t the data stream includes $|\mathcal{D}^t|$ observations, the period is reduced to $R = \frac{\Phi}{4\alpha \times |\mathcal{D}^t|}$ as there are $|\mathcal{D}^t|$ updates every t. Thus, for example, a `StreamCore` of size $|\mathcal{S}| = 400$ can be updated after receiving data over 50 periods when computing R.

The solution maintains a total cost of $O(|\mathcal{S}| \times |\mathcal{D}^t| \times T)$, unlike the squared cost required by having to merge operation of multiple subsets, while ensuring a constant and guaranteed 20-approximation, as a linear cost update exists for an 2-approximation algorithm. To perform calibration using the `StreamCore` $\mathcal{S}$ after R steps, the built small model is first frozen for all blocks except the last one and then calibrated using $\mathcal{S}$. This condition prevents catastrophic forgetting and maintains model performance.

4 Experiments

4.1 Experimental Setup

4.1.1 Tasks and Data Sets. We consider four different tasks: forecasting, imputation, outlier detection and classification. For forecasting and imputations tasks we use popular forecasting data sets [77], namely *ETTh1*, *ETTh2*, *ETTm1*, and *ETTm2*, which measure Electricity Transformer Temperature at different frequencies, and *Weather*, for meteorological data. For outlier detection, we use the *SMD* data set [55] for web server monitoring, *MSL* [28] for spatial exploration, *SMAP* [28] for soil moisture, and *SWaT* [42] for water treatment. We evaluate classification using the UCR time-series archive [15].

4.1.2 Metrics. To evaluate performance on forecasting and imputations tasks, we consider Mean Squared Error (*MSE*). For the outlier detection task, we use the F1 Score metric, with a self-determined threshold [22]. For classification, we evaluate and rank *Accuracy*.

4.1.3 Baselines. To evaluate the effectiveness of the the proposal, we compare it with the following baselines from three different categories. First, as specialized time-series models, we consider `Autoformer` [67], `TimesNet` [66], `PatchTST` [47], `DCdetector` [72], `AnomalyTransformer` [70], `InceptionTime` [18], and `ResNet` [62]. As foundational time-series models, we include `TimeMixer` [58], `Moment` [23], `SymTime` [59], and `Tiny Time Mixers (TTM)` [17]. And finally, `Lag-Llama` [52], as a LLM-based model.

4.2 Experimental Results

For the following experiments, we present the average results, while detailed results are available in the Appendix A.6.

4.2.1 Inference-Time Model for Forecasting. To compare our proposal with other baselines, we first evaluate the inference-time model performance, without any fine-tuning or calibration. The setting follows common practices [17, 66], where each model, the baselines and our proposal, is trained on a large group of data sets and then tested on one unseen data set. For example, when we pre-train on the data sets *ETTh1*, *ETTh2*, *ETTm1*, and *ETTm2*, we then only test on the unseen *Weather* data set.

When comparing the average inference-time model performance (across 96, 192, 336, and 720 horizons) in Table 1, `TimeBlocks` achieves the best results with a substantial margin over all the baselines. In most cases, the differences show statistical significance for t-tests. The large improvements over the large pre-trained models is evidence of the flexibility of selecting the most suitable blocks, rather than relying on a one-fits-all approach that cannot accommodate inference time evaluation and automatic context length selection.

4.2.2 Continual Forecasting. In a continual learning scenario, where the inference-time model is deployed in a data stream, we consider a setting in which data arrives continuously and needs to be accumulated in a buffer for periodic fine-tuning. To establish baselines, we accumulate a random 5% of the incoming data, as this is common when calibrating static models [79]. Similarly, the proposed `StreamCore` also retains only 5% of the data. For consistency, we calibrate all models at the same number of steps $R = 10$, which is below the result presented in Section 3.3, to enable several low-cost calibrations.

Table 1: Forecasting Inference-Time Model (Average MSE).

Model	*ETTh1*	*ETTh2*	*ETTm1*	*ETTm2*	*Weather*
`Autoformer`	1.009	0.587	0.823	0.502	0.353
`PatchTST`	0.675	0.385	0.629	0.293	0.250
`TimeMixer`	0.716	0.373	0.585	0.302	0.238
`TimesNet`	0.792	0.416	0.738	0.330	0.257
`Lag-Llama`	0.750	0.419	0.660	0.345	0.260
`Moment`	0.512	0.655	0.729	0.710	0.732
`SymTime`	0.414	0.365	0.356	0.265	0.234
`TTM`	0.428	0.363	0.545	0.302	0.236
`TimeBlocks`	**0.416**	**0.338**	**0.344**	**0.242**	**0.220**

When comparing the performance of the model after calibrating it in a data stream, as shown in Table 2, `TimeBlocks` consistently achieves the best average results across all settings. The results also demonstrate a notable improvement in the baselines, compared to the inference-time model setting shown in Table 1. This highlights the strong dependence of these models on the data on which they were trained. Therefore, their performance is affected markedly when used in a continual learning setting when new data is arriving. In contrast, the improvements in `TTM` and `TimeBlocks` after calibration are relatively restrained. This suggests that these models perform well in the inference-time mode already, with `TimeBlocks` achieving a much more substantial performance improvement.

Table 2: Forecasting Continual Model (Average MSE).

Model	*ETTh1*	*ETTh2*	*ETTm1*	*ETTm2*	*Weather*
`Autoformer`	0.529	0.520	0.587	0.334	0.336
`PatchTST`	0.464	0.385	0.374	0.280	0.240
`TimeMixer`	0.461	0.387	0.381	0.278	0.233
`TimesNet`	0.457	0.404	0.406	0.289	0.256
`Lag-Llama`	0.459	0.398	0.420	0.283	0.248
`Moment`	0.462	0.602	0.718	0.701	0.723
`TTM`	0.429	0.361	0.440	0.280	0.232
`TimeBlocks + Random`	0.404	0.337	0.348	0.249	0.225
`TimeBlocks`	**0.393**	**0.334**	**0.342**	**0.244**	**0.220**

4.2.3 Coreset Types. In order to evaluate the performance of the coreset strategy, we compared it with random selection in the last two rows of Table 2. The results show that the performance is consistently better for `StreamCore`, as it is able to maintain a better subset for the calibration. The random strategy does not provide a tight guarantee [49], so there may be cases where performance does not improve due to a sub-optimal selection. We do not evaluate other strategies since they rely primarily on computing gradients [5, 10], which is not applicable to an inference-time built model.

4.2.4 Inference-Time Model for Imputation. To assess the flexibility of our proposal, we study its performance on imputation tasks, where a portion of the data set is missing and the goal is to reconstruct the input. In this evaluation, we use an inference-time model setting, using the same set of blocks, and to ensure a fair comparison of the baselines, we have trained these models on imputation tasks. The average results among three three levels of missing data (10%, 20%, and 30%) are shown in Table 3. `TimeBlocks` demonstrates a consistently strong performance across the data

sets. This highlights its flexibility in handling tasks other than forecasting, especially considering that the baselines were specifically trained for imputation tasks.

Table 3: Imputation Inference-Time Model (Average *MSE*).

Model	*ETTh1*	*ETTh2*	*ETTm1*	*ETTm2*	*Weather*
Autoformer	0.890	0.468	0.630	0.395	0.365
PatchTST	0.567	0.334	0.421	0.308	0.281
TimeMixer	0.715	0.348	0.440	0.301	0.269
TimesNet	0.650	0.354	0.486	0.326	0.286
Lag-Llama	0.627	0.338	0.440	0.300	0.266
Moment	0.759	0.404	0.551	0.363	0.330
TTM	0.527	0.300	0.394	0.251	0.245
TimeBlocks	**0.518**	**0.298**	**0.385**	**0.248**	**0.234**

4.2.5 Inference-Time Model for Outlier Detection. The results for the inference-time outlier detection model are shown in Table 4. TimeBlocks achieves better performance than most of the baselines. This highlights the flexibility of the method in handling tasks other than forecasting and imputation, a capability that is lacking in the baselines, such as the otherwise competitive TTM.

Table 4: Outlier Detection Inference-Time Model (F1 Score).

Model	*SMD*	*MSL*	*SMAP*	*SWaT*
Autoformer	0.746	0.740	0.809	0.827
PatchTST	0.823	0.818	0.819	0.822
TimeMixer	0.738	0.744	0.809	0.901
TimesNet	0.813	0.861	0.864	0.879
Lag-Llama	0.789	0.785	0.767	0.767
AnomalyTran	0.836	0.851	0.865	0.894
DCdetector	0.807	**0.875**	0.862	0.914
Moment	0.749	0.727	0.735	0.815
TTM	0.783	0.787	0.762	0.883
TimeBlocks	**0.854**	0.848	**0.866**	**0.915**

4.2.6 Inference-Time Model for Classification. The results for classification using an inference-time model are shown in Figure 7, including only the baselines capable of handling this type of task. The figure, which uses a critical difference rank, shows that TimeBlocks achieves equivalent *Accuracy* performance to the task-specific baseline on the complete data set.

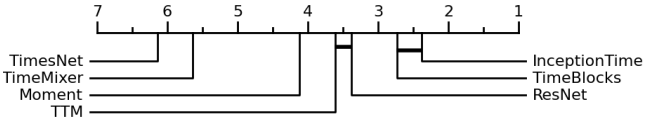
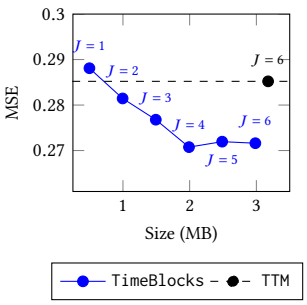


Figure 7: Classification Ranking (Accuracy).

4.2.7 Model Size. When comparing the average model size among data sets, see Figure 8(a), TimeBlocks is the smallest model, making it suitable for deployment in environments with hardware limitations. This result is expected given the design strategy focusing on building a model with only the necessary blocks for a given setting, instead of using general large models. The improvement in terms of size with respect to TTM is slight due to their use of similar small architectures. However, TTM has a fixed design with six blocks, while TimeBlocks allows for the use of fewer blocks than this default, effectively reducing its size while maintaining better performance than TTM, as shown in Figure 8(b). Thus, it is possible for TimeBlocks to achieve better performance than TTM by using $J = 2$ blocks with one third of the TTM size, as shown in Figure 8(a) when $J = 2$.

Model	*Size (MB)*
Autoformer	2449.604
PatchTST	1920.776
TimeMixer	25.476
TimesNet	241.680
Lag-Llama	576.528
Moment	151.641
AnomalyTran	29.472
TTM	3.176
TimeBlocks $J = 6$	2.985
TimeBlocks $J = 2$	0.995

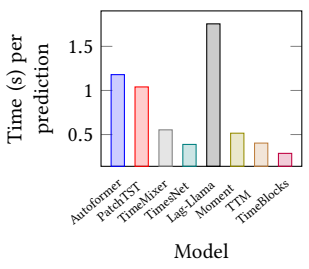
(a) All Baselines.



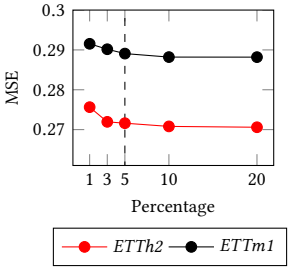
(b) Number of Blocks, *ETTh2*.

Figure 8: Model Size.

4.2.8 Model Efficiency. When evaluating the running time in Figure 9(a), TimeBlocks achieves the lowest inference time, which is due mainly to the small size of the model. This demonstrates the advantage over the baselines, which are bigger and slower, even when the blocks for the model need to be selected.



(a) Inference Time, *ETTh2*.



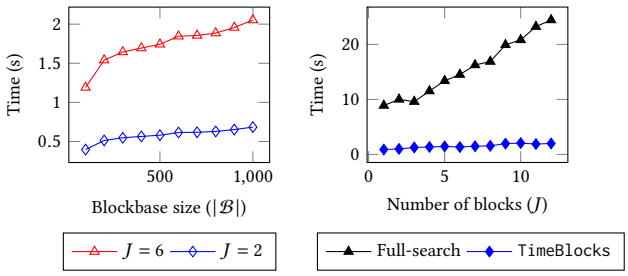
(b) `StreamCore` Size.

Figure 9: Model Efficiency. Forecasting horizon is 96.

Then, in Figure 9(b), we show the effectiveness of the StreamCore for continual calibration across different subset sizes. The results show that using a subset size of more than 5% does not improve performance. Therefore, it becomes a balanced choice for continual calibration, as it clearly improves the performance of smaller subsets.

When evaluating the building time of models of two and six blocks, while adjusting the number of available blocks in the Blockbase, i.e., varying $|\mathcal{B}|$, the scalability of the search process is logarithmic, as shown in Figure 10(a). The search efficiency is achieved by the use of blocks with fingerprints and a router that facilitates their fast retrieval from the Blockbase, as detailed in Section 3.2.5.

The building time of TimeBlocks models is evaluated in Figure 10(b) when varying the number of possible blocks in a model. It is compared with a simple full-search strategy that does not use a router; it is necessary to evaluate all the actual blocks. The results show that the router is effective at maintaining a low execution time with a relatively small growth rate, as discussed in Section 3.2.4.



(a) Scalability, *ETTh2*. **(b) Building Time, *ETTh2*.**

Figure 10: Model Efficiency. Forecasting horizon is 96.

5 Related Work

We discuss the related studies focusing on three main aspects: foundational time-series processing, streaming subset building, and model ensembling.

Foundational Time-Series Processing: Recently, there has been a growing interest in building foundational time-series models to handle multiple data sets and settings, in an attempt to replicate the success of large language models [56] when processing text. We review relevant studies along two dimensions: the support of continual calibration and the model size, as summarized in Table 5.

Table 5: Related Work on Foundational Time-Series Models.

Continual Calibration	*Model Size*	
	Small	*Large*
✓	`TimeBlocks`	-
✗	[16, 17, 58, 60, 61, 68, 76]	[37, 38, 47, 53, 64, 67, 73, 75, 81]

Despite the existence of small models [17, 58, 76], they do not directly support zero-shot processing and require pre-definition of their input size. Also, models that provide zero-shot processing [24, 36, 79] or manage multiple input sizes [2, 64] still rely on very large models. `TimeBlocks` addresses both issues by enabling the efficient construction of models with an automatic selection of input size.

Streaming Subset Building: The concept of compressing a data set into a representative subset, usually referred to as a coreset, has been extensively studied over the years [19, 20, 41, 49]. More recently, there has been a growing interest in building such coresets for scenarios involving data streams [4–6, 33, 35, 46] as it allows for the maintenance of a representation of the stream without requiring additional memory space. These proposals have the drawback that they are specifically designed for particular problems, such as Support Vector Machines [57], making them not suitable for other models, while others do not provide tight guarantees [49] or are not efficient for maintaining the coreset updated [35]. Thus, our proposal addresses all of these issues by introducing a low-cost coreset that only depends on the data stream, not the model.

Model Ensembling: The concept of combining models as a way to improve the performance of machine learning models has been studied for a long time. For example, ensembles [7, 8] and learned indexes [32] have been used for combining the results of several independent models. Approaches such as boosting [54] and mixture-of-experts [3] use the existing results of a model to establish the training of a new model. More recently, post hoc ensembles [50] and routing experts [80] select a combination of the best-performing models after training them on a particular task. `TimeBlocks` differs from these methods by utilizing small pre-trained blocks that iteratively build a model. This approach eliminates the need for additional training while maintaining a small memory footprint.

6 Conclusion and Future Work

The paper introduces `TimeBlocks`, a novel and efficient paradigm for building versatile, inference-assembled time-series models. By using small blocks pre-trained on various time-series tasks and data sets, `TimeBlocks` proposes a routing strategy to determine the most appropriate approach for processing a given time series during inference time. This includes selecting the optimal input size and choosing the most suitable blocks for processing a given time series. Additionally, `StreamCore`, a subset with approximation guarantees, is maintained under a data stream to periodically calibrate the model and ensure its reliability with minimal update cost. Experimental results demonstrate the effectiveness of the method for zero-shot time-series processing and calibration under a data stream, while handling different tasks.

In future work, it is of interest to extend the `Blockbase` to support a wider variety of blocks and compile statistics on their usage. This enhancement would enable support for more specialized models, evaluate quality of blocks, and improve scalability by optimizing both the size and usability of the `Blockbase`.

References

[1] Abdul Fatir Ansari, Lorenzo Stella, Ali Caner Türkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda-Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Bernie Wang. 2024. Chronos: Learning the Language of Time Series. *Trans. Mach. Learn. Res.* 2024 (2024).

[2] Arjun Ashok, Étienne Marcotte, Valentina Zantedeschi, Nicolas Chapados, and Alexandre Drouin. 2024. TACTiS-2: Better, Faster, Simpler Attentional Copulas for Multivariate Time Series. In *ICLR*.

[3] Ran Avnimelech and Nathan Intrator. 1999. Boosted Mixture Of Experts: An Ensemble Learning Scheme. *Neural Comput.* 11, 2 (1999), 483–497.

[4] Vladimir Braverman, Dan Feldman, Harry Lang, Daniela Rus, and Adiel Statman. 2023. Least-Mean-Squares Coresets for Infinite Streams. *IEEE Trans. Knowl. Data Eng.* 35, 9 (2023), 8699–8712.

[5] Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. 2020. Dark Experience for General Continual Learning: a Strong, Simple Baseline. In *NeurIPS*.

[6] David Campos, Bin Yang, Tung Kieu, Miao Zhang, Chenjuan Guo, and Christian S. Jensen. 2024. QCore: Data-Efficient, On-Device Continual Calibration for Quantized Models. *Proc. VLDB Endow.* 17, 11 (2024), 2708–2721.

[7] David Campos, Miao Zhang, Bin Yang, Tung Kieu, Chenjuan Guo, and Christian S. Jensen. 2023. LightTS: Lightweight Time Series Classification with Adaptive Ensemble Distillation. *Proc. ACM Manag. Data* 1, 2 (2023), 171:1–171:27.

[8] Rich Caruana, Alexandru Niculescu-Mizil, Geoff Crew, and Alex Ksikes. 2004. Ensemble selection from libraries of models. In *ICML*, Vol. 69.

[9] Cristian Challu, Kin G. Olivares, Boris N. Oreshkin, Federico Garza Ramírez, Max Mergenthaler Canseco, and Artur Dubrawski. 2023. NHITS: Neural Hierarchical Interpolation for Time Series Forecasting. In *AAAI*. 6989–6997.

[10] Arslan Chaudhry, Marc'Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. 2019. Efficient Lifelong Learning with A-GEM. In *ICLR*.

[11] Sang Keun Choe, Hwijeen Ahn, Juhan Bae, Kewen Zhao, Minsoo Kang, Youngseog Chung, Adithya Pratapa, Willie Neiswanger, Emma Strubell, Teruko Mitamura, Jeff G. Schneider, Eduard H. Hovy, Roger B. Grosse, and Eric P. Xing. 2024. What is Your Data Worth to GPT? LLM-Scale Data Valuation with Influence Functions. *CoRR* abs/2405.13954 (2024).

[12] Ben Cohen, Emaad Khwaja, Kan Wang, Charles Masson, Elise Ramé, Youssef Doubli, and Othmane Abou-Amal. 2024. Toto: Time Series Optimized Transformer for Observability. *CoRR* abs/2407.07874 (2024).

[13] Vincent Cohen-Addad, Niklas Hjuler, Nikos Parotsidis, David Saulpic, and Chris Schwiegelshohn. 2019. Fully Dynamic Consistent Facility Location. In *NeurIPS*. 3250–3260.

[14] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. 2024. A Decoder-only Foundation Model for Time-series Forecasting. In *ICML*, Vol. 235. 10148–10167.

[15] Hoang Anh Dau, Anthony J. Bagnall, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, and Eamonn J. Keogh. 2019. The UCR time series archive. *IEEE CAA J. Autom. Sinica* 6, 6 (2019), 1293–1305.

[16] Vijay Ekambaram, Arindam Jati, Nam Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. TSMixer: Lightweight MLP-Mixer Model for Multivariate Time Series Forecasting. In *KDD*. 459–469.

[17] Vijay Ekambaram, Arindam Jati, Nam H. Nguyen, Pankaj Dayama, Chandra Reddy, Wesley M. Gifford, and Jayant Kalagnanam. 2024. Tiny Time Mixers (TTMs): Fast Pre-trained Models for Enhanced Zero/Few-Shot Forecasting of Multivariate Time Series. In *NeurIPS*.

[18] Hassan Ismail Fawaz, Benjamin Lucas, Germain Forestier, Charlotte Pelletier, Daniel F. Schmidt, Jonathan Weber, Geoffrey I. Webb, Lhassane Idoumghar, Pierre-Alain Muller, and François Petitjean. 2020. InceptionTime: Finding AlexNet for time series classification. *Data Min. Knowl. Discov.* 34, 6 (2020), 1936–1962.

[19] Dan Feldman and Michael Langberg. 2011. A unified framework for approximating and clustering data. In *STOC*. 569–578.

[20] Dan Feldman and Tamir Tassa. 2015. More Constraints, Smaller Coresets: Constrained Matrix Approximation of Sparse Big Data. In *SIGKDD*. 249–258.

[21] Milton Friedman. 1940. A Comparison of Alternative Tests of Significance for the Problem of m Rankings. *Annals of Mathematical Statistics* 11 (1940), 86–92.

[22] Mononito Goswami, Cristian I. Challu, Laurent Callot, Lenon Minorics, and Andrey Kan. 2023. Unsupervised Model Selection for Time Series Anomaly Detection. In *ICLR*.

[23] Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. 2024. MOMENT: A Family of Open Time-series Foundation Models. In *ICML*.

[24] Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. 2023. Large Language Models Are Zero-Shot Time Series Forecasters. In *NeurIPS*.

[25] Chenjuan Guo, Christian S. Jensen, and Bin Yang. 2014. Towards Total Traffic Awareness. *SIGMOD Rec.* 43, 3 (2014), 18–23.

[26] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[27] J. A. Hoogeveen. 1991. Analysis of Christofides' heuristic: Some paths are more difficult than cycles. *Oper. Res. Lett.* 10, 5 (1991), 291–295.

[28] Kyle Hundman, Valentino Constantinou, Christopher Laporte, Ian Colwell, and Tom Söderström. 2018. Detecting Spacecraft Anomalies Using LSTMs and Nonparametric Dynamic Thresholding. In *SIGKDD*. 387–395.

[29] Xin Jin and Jiawei Han. 2010. *K-Means Clustering*. Springer US, Boston, MA, 563–564.

[30] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Trans. Big Data* 7, 3 (2021), 535–547.

[31] Andrej Karpathy. 2019. Multi-task learning in the wilderness. In *ICML*. https://slideslive.com/38917690/multitask-learning-in-the-wilderness

[32] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *SIGMOD*. 489–504.

[33] Max Dupré la Tour, Monika Henzinger, and David Saulpic. 2024. Fully Dynamic k-Means Coreset in Near-Optimal Update Time. In *ESA (LIPIcs, Vol. 308)*. 100:1–100:16.

[34] Yuening Li, Zhengzhang Chen, Daochen Zha, Mengnan Du, Jingchao Ni, Denghui Zhang, Haifeng Chen, and Xia Hu. 2022. Towards Learning Disentangled Representations for Time Series. In *SIGKDD*. 3270–3278.

[35] Yiming Li, Yanyan Shen, and Lei Chen. 2022. Camel: Managing Data for Efficient Stream Learning. In *SIGMOD*. 1271–1285.

[36] Zhe Li, Xiangfei Qiu, Peng Chen, Yihang Wang, Hanyin Cheng, Yang Shu, Jilin Hu, Chenjuan Guo, Aoying Zhou, Christian S. Jensen, and Bin Yang. 2025. TSFM-Bench: A Comprehensive and Unified Benchmark of Foundation Models for Time Series Forecasting. In *KDD*. 5595–5606.

[37] Jiaming Liang, Lei Cao, Samuel Madden, Zack Ives, and Guoliang Li. 2024. RITA: Group Attention is All You Need for Timeseries Analytics. *Proc. ACM Manag. Data* 2, 1 (2024), 62:1–62:28.

[38] Zhiyu Liang, Jianfeng Zhang, Chen Liang, Hongzhi Wang, Zheng Liang, and Lujia Pan. 2023. A Shapelet-based Framework for Unsupervised Multivariate Time Series Representation Learning. *Proc. VLDB Endow.* 17, 3 (2023), 386–399.

[39] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *ICLR*.

[40] Hua Lu, Bin Yang, and Christian S. Jensen. 2011. Spatio-temporal joins on symbolic indoor tracking data. In *ICDE*. 816–827.

[41] Alaa Maalouf, Gilad Eini, Ben Mussay, Dan Feldman, and Margarita Osadchy. 2024. A Unified Approach to Coreset Learning. *IEEE Trans. Neural Networks Learn. Syst.* 35, 5 (2024), 6893–6905.

[42] Aditya P. Mathur and Nils Ole Tippenhauer. 2016. SWaT: a water treatment testbed for research and training on ICS security. In *CySWater@CPSWeek*. 31–36.

[43] Nimrod Megiddo and Kenneth J. Supowit. 1984. On the Complexity of Some Common Geometric Location Problems. *SIAM J. Comput.* 13, 1 (1984), 182–196.

[44] Adam Meyerson. 2001. Online Facility Location. In *FOCS*. 426–431.

[45] Baharan Mirzasoleiman, Jeff A. Bilmes, and Jure Leskovec. 2020. Coresets for Data-efficient Training of Machine Learning Models. In *ICML*, Vol. 119. 6950–6960.

[46] Guanlin Mo, Shihong Song, and Hu Ding. 2024. Towards Metric DBSCAN: Exact, Approximate, and Streaming Algorithms. *Proc. ACM Manag. Data* 2, 3 (2024), 178.

[47] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *ICLR*.

[48] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. 2020. N-BEATS: Neural Basis Expansion Analysis for Interpretable Time Series Forecasting. In *ICLR*.

[49] Jeff M. Phillips. 2017. Coresets and Sketches. In *Handbook of Discrete and Computational Geometry, Third Edition*, Csaba D. Toth, Joseph O'Rourke, and Jacob E. Goodman (Eds.). Chapman and Hall/CRC, New York, 1269–1288.

[50] Lennart Oswald Purucker, Lennart Schneider, Marie Anastacio, Joeran Beel, Bernd Bischl, and Holger H. Hoos. 2023. Q(D)O-ES: Population-based Quality (Diversity) Optimisation for Post Hoc Ensemble Selection in AutoML. In *AutoML (Proceedings of Machine Learning Research, Vol. 224)*. 10/1–34.

[51] Xiangfei Qiu, Jilin Hu, Lekui Zhou, Xingjian Wu, Junyang Du, Buang Zhang, Chenjuan Guo, Aoying Zhou, Christian S. Jensen, Zhenli Sheng, and Bin Yang. 2024. TFB: Towards Comprehensive and Fair Benchmarking of Time Series Forecasting Methods. *Proc. VLDB Endow.* 17, 9 (2024), 2363–2377.

[52] Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Arian Khorasani, George Adamopoulos, Rishika Bhagwatkar, Marin Bilos, Hena Ghonia, Nadhir Vincent Hassen, Anderson Schneider, Sahil Garg, Alexandre Drouin, Nicolas Chapados, Yuriy Nevmyvaka, and Irina Rish. 2023. Lag-Llama: Towards Foundation Models for Time Series Forecasting. In *NeurIPS*.

[53] Lei Rui, Xiangdong Huang, Shaoxu Song, Yuyuan Kang, Chen Wang, and Jianmin Wang. 2024. Time Series Representation for Visualization in Apache IoTDB. *Proc. ACM Manag. Data* 2, 1 (2024), 35:1–35:26.

[54] Robert E. Schapire. 2003. *The Boosting Approach to Machine Learning: An Overview*. Springer, New York, 149–171.

[55] Ya Su, Youjian Zhao, Chenhao Niu, Rong Liu, Wei Sun, and Dan Pei. 2019. Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network. In *SIGKDD*. 2828–2837.

[56] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *CoRR* abs/2302.13971 (2023).

[57] Murad Tukan, Cenk Baykal, Dan Feldman, and Daniela Rus. 2021. On coresets for support vector machines. *Theor. Comput. Sci.* 890 (2021), 171–191.

[58] Shiyu Wang, Haixu Wu, Xiaoming Shi, Tengge Hu, Huakun Luo, Lintao Ma, James Y. Zhang, and Jun Zhou. 2024. TimeMixer: Decomposable Multiscale Mixing for Time Series Forecasting. In *ICLR*.

[59] Wenxuan Wang, Kai Wu, Yujian Betterest Li, Dan Wang, and Xiaoyu Zhang. 2025. Synthetic Series-Symbol Data Generation for Time Series Foundation Models. In *NeurIPS*.

[60] Yihang Wang, Yuying Qiu, Peng Chen, Yang Shu, Zhongwen Rao, Lujia Pan, Bin Yang, and Chenjuan Guo. 2025. LightGTS: A Lightweight General Time Series Forecasting Model. In *ICML*.

[61] Yihang Wang, Yuying Qiu, Peng Chen, Kai Zhao, Yang Shu, Zhongwen Rao, Lujia Pan, Bin Yang, and Chenjuan Guo. 2025. Towards a General Time Series Forecasting Model with Unified Representation and Adaptive Transfer. In *ICML*.

[62] Zhiguang Wang, Weizhong Yan, and Tim Oates. 2017. Time series classification from scratch with deep neural networks: A strong baseline. In *IJCNN*. 1578–1585.

[63] Frank Wilcoxon. 1945. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1, 6 (1945), 80–83.

[64] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2024. Unified Training of Universal Time Series Forecasting Transformers. In *ICML*, Vol. 235. 53140–53164.

[65] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2024. Unified Training of Universal Time Series Forecasting Transformers. In *ICML (Proceedings of Machine Learning Research)*. 53140–53164.

[66] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2023. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. In *ICLR*.

[67] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. In *NeurIPS*. 22419–22430.

[68] Xingjian Wu, Jianxin Jin, Wanghui Qiu, Peng Chen, Yang Shu, Bin Yang, and Chenjuan Guo. 2026. Aurora: Towards Universal Generative Multimodal Time Series Forecasting. In *ICLR*.

[69] Xingjian Wu, Xiangfei Qiu, Hongfan Gao, Jilin Hu, Bin Yang, and Chenjuan Guo. 2025. K2VAE: A Koopman-Kalman Enhanced Variational AutoEncoder for Probabilistic Time Series Forecasting. In *ICML*.

[70] Jiehui Xu, Haixu Wu, Jianmin Wang, and Mingsheng Long. 2022. Anomaly Transformer: Time Series Anomaly Detection with Association Discrepancy. In *ICLR*.
[71] Sean Bin Yang, Jilin Hu, Chenjuan Guo, Bin Yang, and Christian S. Jensen. 2023. LightPath: Lightweight and Scalable Path Representation Learning. In *KDD*. 2999–3010.
[72] Yiyuan Yang, Chaoli Zhang, Tian Zhou, Qingsong Wen, and Liang Sun. 2023. DCdetector: Dual Attention Contrastive Representation Learning for Time Series Anomaly Detection. In *KDD*. 3033–3045.
[73] Zhihan Yue, Yujing Wang, Juanyong Duan, Tianmeng Yang, Congrui Huang, Yunhai Tong, and Bixiong Xu. 2022. TS2Vec: Towards Universal Representation of Time Series. In *AAAI*. 8980–8987.
[74] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are Transformers Effective for Time Series Forecasting?. In *AAAI*. 11121–11128.
[75] George Zerveas, Srideepika Jayaraman, Dhaval Patel, Anuradha Bhamidipaty, and Carsten Eickhoff. 2021. A Transformer-based Framework for Multivariate Time Series Representation Learning. In *SIGKDD*. 2114–2124.
[76] Shuhan Zhong, Sizhe Song, Weipeng Zhuo, Guanyao Li, Yang Liu, and S.-H. Gary Chan. 2024. A Multi-Scale Decomposition MLP-Mixer for Time Series Analysis. *Proc. VLDB Endow.* 17, 7 (2024), 1723–1736.
[77] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. In *AAAI*. 11106–11115.
[78] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. 2022. FEDformer: Frequency Enhanced Decomposed Transformer for Long-term Series Forecasting. In *ICML*, Vol. 162. 27268–27286.
[79] Tian Zhou, Peisong Niu, Xue Wang, Liang Sun, and Rong Jin. 2023. One Fits All: Power General Time Series Analysis by Pretrained LM. In *NeurIPS*.
[80] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Y. Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. 2022. Mixture-of-Experts with Expert Choice Routing. In *NeurIPS*.
[81] Rundong Zuo, Guozhong Li, Rui Cao, Byron Choi, Jianliang Xu, and Sourav S. Bhowmick. 2024. DARKER: Efficient Transformer with Data-driven Attention Mechanism for Time Series. *Proc. VLDB Endow.* 17, 11 (2024), 3229–3242.

A Appendix

A.1 Blocks Pre-Training

The process starts when receiving a pair of a time series with its corresponding label, along with the number of blocks required, as shown in line 1. In this pair, x represents the time-series data in a window with a specific context length, while the label y is the window of the next observed values over a specific horizon in forecasting. In imputation and outlier detection tasks, y denotes the window of observed values with the same context length, while in classification y is a label. To build a model, P blocks from a given architecture (e.g., MLP, LSTM, or Attention) are selected using their default configuration, and following a decreasing scale pattern (see Section 3.2.2), as shown in lines 4–6. Then, the model is trained minimizing a loss as shown in line 7. Once the training is complete, the model and its blocks are returned.

Algorithm A1 Blocks Training.

1: **Input**: Time series pair (x, y), number of blocks P
2: **Output**: Model, model blocks
3: $Model \leftarrow \emptyset$
4: **for** p in $1 \dots P$ **do**
5: $\quad B_{new} \leftarrow$ New block with 2^{P-p} sampling.
6: $\quad Model \leftarrow$ Attach B_{new}
7: Minimize $Loss(Model(x), y)$ ▷ *Model training*
8: **return** *Model*, *Model.Blocks*

A.2 Router Training

Having a defined router and model, this process evaluates the model and records the residual of all blocks r_p, as shown in line 3. Then, for each block of the model, it obtains the corresponding fingerprint (line 5) and trains the router to approximate $Router(r_p) \approx b^1_{p+1}$, considering both the fingerprint and the output from the previous block, as shown in line 7. After iterating through all blocks, the router is returned to be trained with other end-to-end models.

Algorithm A2 Router Training.

1: **Input**: $Router(\cdot)$, $Model(\cdot)$
2: **Output**: Updated $Router(\cdot)$
3: $\{r_1, \dots, r_P\} \leftarrow$ Evaluate *Model* ▷ *Record residuals*
4: **for** B_p in *Model* **do** ▷ *Iterate the blocks of the model*
5: $\quad b^1_p \leftarrow$ Get fingerprint from B_p
6: $\quad$ **for** $e \leftarrow 1, \dots, E$ **do** ▷ *Epochs*
7: $\quad\quad$ Minimize $Loss(Router(r_{p-1}), b^1_p)$
8: **return** $Router(\cdot)$

A.3 Block Selection

Given a trained `Blockbase` and a new time series, the algorithm first evaluates the time series across different window sizes and then selects the most suitable cluster by comparing the their fingerprints, as shown in lines 5–9. Then, after selecting the processing cluster in line 10, the time series is split accordingly and is standardized as explained in Section 2.2. Then, to add a new block B_j, the model is evaluated until the last selected block B_{j-1}, recording the associated residual r_{j-1}, as shown in line 15. The residual is then evaluated by the router to compute the fingerprint for a candidate block in line 16. The fingerprint is compared with the blocks within the cluster to find the most similar one by using vector similarity search [11, 30], as shown in lines 17–18. Finally, the identified block is attached to the model. Having repeated the process to obtain up to J blocks, the inference-time model is returned.

A.4 StreamCore Detailed Derivations

Consider the following two properties:
Weight Relaxation: Assume there is a mapping $\gamma : \mathcal{D} \to \mathcal{S}$ that assigns every data point $i \in \mathcal{D}$ to one $j \in \mathcal{S}$, therefore $\gamma(i) = j$.
Lipschitz Continuous: A function $f : X \to Y$ is λ–Lipschitz continuous if exists a constant $\lambda > 0$ such that for any $x_1, x_2 \in X$, $\|f(x_1) - f(x_2)\| \leq \lambda \|x_1 - x_2\|$.

Then, considering weight relaxation, the sums of Equation 1 in the main paper can be written as follows.

$$\begin{aligned}
&\left\| \sum_{i \in \mathcal{D}}^{n} l(x_i, f(x_i)) - \sum_{j \in \mathcal{S}}^{n} w_j l(x_j, f(x_j)) \right\| \\
&= \left\| \sum_{i \in \mathcal{D}}^{n} l(x_i, f(x_i)) - \sum_{i \in \mathcal{D}}^{n} l(x_{\gamma(i)}, f(x_{\gamma(i)})) \right\| \\
&\leq \left\| \sum_{i \in \mathcal{D}}^{n} l(x_i, f(x_i)) - l(x_{\gamma(i)}, f(x_{\gamma(i)})) \right\|, \qquad (3)
\end{aligned}$$

Algorithm A3 Block Selection.

Input: Time series x, `Blockbase`, $Router(\cdot)$, block budget J
Output: Zero-shot *Model*

$Model \leftarrow \emptyset, \quad Distances \leftarrow \emptyset$
$Clusters \leftarrow$ Extract from `Blockbase`

for *Clus* in *Clusters* **do**
  $x_{st} \leftarrow$ Split x in windows of size *Clus.ContextLength*
  $r_0 \leftarrow$ Standardization (x_{st}) ▷ *Aligned with Sec. 3.2.2*
  $\hat{B}^1_{cand} \leftarrow Router(r_0)$ ▷ *Estimate candidate fingerprint*
  $Distances[Clus] \leftarrow Distance(\hat{B}^1_{cand}, Clus.Fingerprint)$
$minCluster \leftarrow \arg\min Distances$
$x_{st} \leftarrow$ Split x in windows of size of *minCluster* center
$r_0 \leftarrow$ Standardization (x_{st}) ▷ *Aligned with Sec. 3.2.2*

for j in $1, \ldots, J$ **do**
  if $j > 1$ **then**
    $r_{j-1} \leftarrow$ Evaluate $Model(x_{st})$ ▷ *Register residual*
  $\hat{B}^1_j \leftarrow Router(r_{j-1})$ ▷ *Router estimates candidate fingerprint*
  for $\{B_1, \ldots, B_P\}$ in *minCluster* **do** ▷ *Blocks in the cluster*
    $Distances[B_p] \leftarrow Distance(\hat{B}^1_j, B_p)$ ▷ *Similarity search*
  $minBlock \leftarrow \arg\min Distances$
  $Model \leftarrow$ Attach *minBlock*
return *Model*

where the last part of Equation 3 is due to the triangle inequality and will be minimal when each point is assigned to the $j \in \mathcal{S}$ with the closest value. Therefore, we have

$$\left\| \sum_{i \in \mathcal{D}}^{n} l(x_i, f(x_i)) - \sum_{j \in \mathcal{S}}^{n} w_j l(x_j, f(x_j)) \right\| \leq \left\| \sum_{i \in \mathcal{D}}^{n} \min_{j \in \mathcal{S}} l(x_i, f(x_i)) - l(x_j, f(x_j)) \right\| \quad (4)$$

Assuming that both $f(\cdot)$ and $l(\cdot, \cdot)$ are Lipschitz Continuous, since f is a deep learning based model and l is a loss function, as derived by previous studies [35], the right-hand side of Equation 4 can be bounded as follows.

$$\begin{aligned} \|l(x_i, f(x_i)) - l(x_j, f(x_j))\| &\leq \lambda_1 \|f(x_i) - f(x_j)\| \\ &\leq \lambda_1 \lambda_2 \|x_i - x_j\| = \lambda \|x_i - x_j\| \end{aligned} \quad (5)$$

Applying the results of Equations 4 and 5 to Equation 1, we get:

$$\frac{1}{n} \left\| \sum_{i \in \mathcal{D}}^{n} l(x_i, f(x_i)) - \sum_{j \in \mathcal{S}}^{n} w_j l(x_j, f(x_j)) \right\| \leq \frac{1}{n} \sum_{i \in \mathcal{D}}^{n} \min_{j \in \mathcal{S}} \lambda \|x_i - x_j\|$$

which is a bounded approximation for the coreset.

For computing the period R, it exists an $(8\alpha + 4)$-approximation with $R = \frac{\Phi}{4\alpha}$ updates [13]. Then, as each time t the data stream includes $|\mathcal{D}^t|$ observations, the period is reduced to $R = \frac{\Phi}{4\alpha \times |\mathcal{D}^t|}$ as there are $|\mathcal{D}^t|$ updates every t. This means, for example, that a `StreamCore` of size $|\mathcal{S}| = 400$ can be updated after receiving data over 50 periods, as computed below.

$$R = \frac{\Phi}{4\alpha \times |\mathcal{D}^t|} = \frac{|\mathcal{S}| \times |\mathcal{D}^t|}{4\alpha \times |\mathcal{D}^t|} = \frac{400}{4 \times 2} = 50$$

In the context of time-series data, a concept drift is treated as an external event that triggers model calibration. For example, when a change in the data stream exceeds a predefined threshold, the model can be calibrated using the `StreamCore`, a component that continuously collects new data, which may or may not include a concept drift. This approach enables a general evaluation, independent of when a concept drift occurs during deployment.

A.5 Detailed Experimental Setup

A.5.1 Tasks and Data Sets. We consider four different tasks: forecasting, imputation, outlier detection, and classification. For forecasting and imputations tasks we use popular forecasting data sets [77], namely *ETTh1*, *ETTh2*, *ETTm1*, and *ETTm2*, which measure Electricity Transformer Temperature at different frequencies, and *Weather*, for meteorological data. For outlier detection, we use data sets related to web server monitoring (*SMD* [55]), spatial exploration (*MSL* [28]), soil moisture (*SMAP* [28]), and water treatment (*SWaT* [42]). These data sets have ground truth labels that indicate outliers. We solely use these labels for evaluating model accuracy, not for model training. Table A1 presents the details for the data sets, including their number of dimensions, training partition, and frequency. For classification, we use the UCR time-series archive [15], which consists of 128 data sets from a wide variety of domains.

For all tasks, the context length for the baselines is set to 96, as it is used most commonly [66, 67, 78], while for pre-training the blocks, we consider a power-of-two set of context lengths varying 2^6 to 2^{11}, i.e., $\{64, 128, 256, 512, 1024, 2048\}$.

A.5.2 Metrics. To evaluate performance on forecasting and imputations tasks, we consider two metrics: Mean Absolute Error (*MAE*) and Mean Squared Error (*MSE*) computed over the testing set after implementing either the inference-time or calibrated model. These metrics are defined as follows.

$$MAE = \frac{1}{n} \sum_{i=1}^{n} |y - \hat{y}| \qquad MSE = \frac{1}{n} \sum_{i=1}^{n} (y - \hat{y})^2$$

For the outlier detection task, we use the F1 Score metric, which is the harmonic mean of precision and recall, with a self-determined threshold [22].

For classification, we generate a critical difference diagram based on *Accuracy* after applying the null-hypothesis Friedman test [21] and the Wilcoxon–Holm post-hoc test [26, 63] to rank the evaluated methods. The diagram is built by computing the difference in performance between the baselines that support classification for each data set and ranking them from the smallest to the largest difference. Then, the average rank across all data sets is calculated, with a thick horizontal line indicating when the difference between the evaluated methods is not statistically significant.

A.5.3 Baselines. To evaluate the effectiveness of the the proposal, we compare it with the following baselines from three different categories to enable a fair comparison among different time-series processing approaches:

Time-Series Specialized Models:

- `Autoformer` [67]: a Transformer-based model that introduces time-series decomposition through an auto-correlation mechanism to handling time-dependency, instead of exclusively relying on default language processing.
- `TimesNet` [66]: introduces a two-dimensional decomposition of time series to measure variations both between periods and within a given period. This approach allows for the use of two-dimensional methods to effectively process time-series data.
- `PatchTST` [47]: a method that segments time series into subseries called patches. This allows for processing them as contextualized tokens rather than as independent observations within a Transformer-based encoding mechanism.
- `AnomalyTransformer` [70]: a popular Transformer-based outlier detection method enhanced by a metric measuring the associativity between adjacent data points.
- `DCdetector` [72]: a representation learning outlier detection method uses multi-scale attention layers to identify weak correlations among data points.
- `InceptionTime` [18]: a convolutional neural network that employs variable-length convolution filters to capture temporal patterns across different timescales.
- `ResNet` [62]: an adaptation of the backbone residual network architecture for time-series classification.

LLM-Based Model:

- `Lag-Llama` [52]: introduces the architecture of an LLM [56] for processing time series while considering the covariance of each observation with respect to its context length.

Time-Series Foundational Models:

- `TimeMixer` [58]: proposes granular time-series decomposition and then mixing the results in order to identify time-series properties such as trends and cycles. This approach, called a mixer, enables more contextualized processing of time series.
- `Tiny Time Mixers (TTM)` [17]: unifies existing studies [16, 76], bringing an adaptive patching and a mixer architecture into an efficient MLP structure to enable efficient time-series processing.
- `Moment` [23]: combines the patching mechanism with a Transformer encoder, which are then trained using a masking strategy to facilitate the processing of multiple tasks.
- `SymTime` [59]: it introduces the pairing of time series with symbolic expressions to overcome data scarcity. It is only evaluated for reference in the forecasting task (i.e. Table 1) due to its parallel release.

Potential baselines such `Moirai` [65], `ToTo` [12], and `Chronos` [1] are not evaluated, as they are specifically designed for time-series forecasting, requiring significant adaptation and retraining to function in other tasks.

A.5.4 Implementation. The proposed method is implemented using Python 3.9.19 and the deep learning framework PyTorch 2.3.1. All models are tested under Ubuntu 22.04.4 using Titan RTX GPUs with 24GB VRAM and an Intel Xeon W-2155 with 128GB RAM. The code is available as supplementary material. When applicable, all methods use the hyper-parameter configurations recommended in their documentation. Otherwise, we adjust them using the validation set, following common practices. For `Lag-Llama` we use the average of 100 samples from its distribution. When training the blocks and the router, we use the AdamW optimizer [39] trained for 50 epochs with a learning rate of 0.001 and a batch size of 64.

To build the inference-time model, we utilize the router to select $J = 6$ blocks (see Algorithm A3) as the default backbone, and we also vary it, as will be discussed in Section A.6.2. This selection is to ensure consistency and comparability in terms of efficiency when compared to the efficient `TTM` baseline. The comparison between the block candidate and the `TimeBlocks` fingerprints is calculated using cosine distance. For all the evaluations, the best results are highlighted in bold, while the second best results are underlined.

Table A1: Data Sets.

	Data Set	*Dims*	*Time Steps*	*Train/Valid/Test*	*Frequency*
Forecasting	*ETTh1*	7	17420	10452/3484/3484	Hours
	ETTh2	7	17420	10452/3484/3484	Hours
	ETTm1	7	69680	41808/13936/13936	Minutes
	ETTm2	7	69680	41808/13936/13936	Minutes
	Weather	21	52696	24388/14154/14154	Hours
Outlier	*SMD*	25	7.18 million	4.31/1.43/1.43 million	Seconds
	MSL	55	132046	58318/36864/36864	Hours
	SMAP	38	562800	135184/213808/213808	Hours
	SWaT	51	489600	293760/97920/97920	Seconds

A.6 Additional Experimental Results

A.6.1 Disaggregated Results. Detailed results for Tables 1, 2, and 3 in the main paper are shown in Tables A3, A4, and A2, respectively.

Table A2: Imputation Inference-Time Model (*MSE*). Detailed results for Table 3.

Model	*Mask*	*ETTh1*	*ETTh2*	*ETTm1*	*ETTm2*	*Weather*
`Autoformer`	10%	0.549	0.365	0.467	0.280	0.326
	20%	0.807	0.488	0.591	0.417	0.346
	30%	1.313	0.552	0.834	0.488	0.423
`PatchTST`	10%	0.427	0.307	0.323	0.246	0.203
	20%	0.618	0.317	0.398	0.260	0.281
	30%	0.655	0.380	0.542	0.416	0.358
`TimeMixer`	10%	0.438	0.294	0.319	0.240	0.201
	20%	0.560	0.375	**0.395**	0.262	0.265
	30%	1.148	0.375	0.605	0.401	0.342
`TimesNet`	10%	0.441	0.324	0.340	0.253	0.223
	20%	0.723	0.326	0.415	0.308	0.279
	30%	0.787	0.411	0.704	0.417	0.356
`Lag-Llama`	10%	0.427	0.290	0.311	0.234	0.205
	20%	0.616	0.324	0.405	0.287	0.272
	30%	0.839	0.399	0.604	0.380	0.322
`Moment`	10%	0.497	0.331	0.404	0.279	0.254
	20%	0.741	0.418	0.501	0.354	0.325
	30%	1.038	0.463	0.749	0.456	0.411
`TTM`	10%	0.417	0.234	0.260	0.186	0.202
	20%	0.539	0.307	0.406	0.258	0.245
	30%	0.627	0.361	0.514	0.307	0.287
`TimeBlocks`	10%	**0.406**	**0.231**	**0.251**	**0.184**	**0.193**
	20%	**0.529**	**0.304**	0.397	**0.256**	**0.240**
	30%	**0.617**	**0.357**	**0.506**	**0.304**	**0.270**

Table A3: Forecasting Inference-Time Model. Detailed results for Table 1.

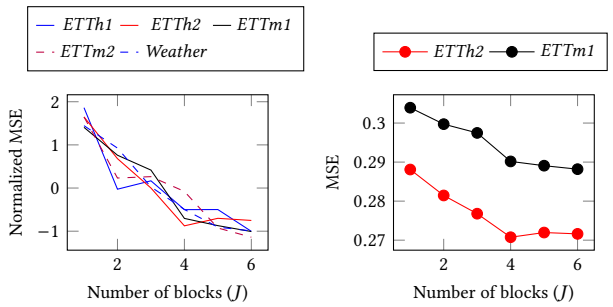
Model		Autoformer		PatchTST		TimeMixer		TimesNet		Lag-Llama		Moment		TTM		TimeBlocks	
Data Set	*Horizon*	*MAE*	*MSE*	*MAE*	*MSE*	*MAE*	*MSE*	*MAE*	*MSE*	*MAE*	*MSE*	*MAE*	*MSE*	*MAE*	*MSE*	*MAE*	*MSE*
ETTh1	96	0.714	1.018	0.521	0.586	0.467	0.500	0.596	0.742	0.557	0.647	0.455	0.471	0.404	**0.365**	**0.397**	0.370
	192	0.638	0.847	0.578	0.719	0.505	0.560	0.600	0.744	0.563	0.703	0.474	0.505	0.429	**0.393**	**0.425**	0.409
	336	0.715	1.022	0.583	0.715	0.543	0.655	0.646	0.846	0.597	0.745	0.491	0.534	0.448	**0.415**	**0.442**	0.429
	720	0.770	1.149	0.582	0.681	0.751	1.148	0.640	0.837	0.638	0.906	0.515	0.538	0.493	0.538	**0.475**	**0.456**
ETTh2	96	0.535	0.532	0.349	0.302	0.345	0.296	0.375	0.334	0.397	0.352	0.519	0.612	0.343	0.285	**0.337**	**0.271**
	192	0.553	0.575	0.395	0.378	0.389	0.375	0.424	0.424	0.409	0.416	0.535	0.649	0.380	0.341	**0.378**	**0.333**
	336	0.579	0.624	0.432	0.424	0.420	0.410	0.447	0.448	0.455	0.458	0.560	0.670	0.404	0.383	**0.401**	**0.355**
	720	0.577	0.616	0.450	0.435	**0.431**	0.409	0.461	0.456	0.441	0.450	0.593	0.689	0.439	0.441	0.434	**0.392**
ETTm1	96	0.618	0.808	0.451	0.516	0.476	0.570	0.540	0.709	0.485	0.612	0.564	0.710	0.357	0.413	**0.343**	**0.288**
	192	0.600	0.772	0.477	0.542	0.478	0.550	0.566	0.736	0.525	0.637	0.576	0.726	0.374	0.476	**0.368**	**0.326**
	336	0.644	0.872	0.543	0.691	0.501	0.605	0.553	0.681	0.515	0.698	0.592	0.738	0.398	0.553	**0.386**	**0.362**
	720	0.637	0.840	0.591	0.767	0.525	0.615	0.599	0.826	0.582	0.693	0.612	0.742	0.417	0.737	**0.413**	**0.401**
ETTm2	96	0.480	0.420	0.272	0.193	0.277	0.201	0.304	0.226	0.330	0.244	0.553	0.695	0.257	0.187	**0.257**	**0.164**
	192	0.482	0.457	0.311	0.254	0.315	0.262	0.356	0.310	0.351	0.318	0.566	0.717	0.302	0.261	**0.295**	**0.216**
	336	0.519	0.524	0.345	0.308	0.351	0.320	0.379	0.349	0.390	0.357	0.574	0.723	0.331	0.323	**0.328**	**0.259**
	720	0.560	0.605	0.412	0.416	0.411	0.423	0.420	0.434	0.433	0.460	0.584	0.703	0.384	0.436	**0.378**	**0.329**
Weather	96	0.340	0.272	0.212	0.168	0.207	0.159	0.219	0.168	0.225	0.184	0.560	0.717	0.202	0.154	**0.194**	**0.145**
	192	0.401	0.352	0.251	0.214	0.249	0.204	0.265	0.224	0.271	0.229	0.572	0.734	0.245	0.203	**0.238**	**0.191**
	336	0.399	0.360	0.295	0.272	0.287	0.257	0.305	0.283	0.318	0.278	0.590	0.743	0.287	0.256	**0.278**	**0.243**
	720	0.431	0.429	0.345	0.347	0.342	0.335	0.354	0.355	0.339	0.348	0.603	0.731	0.348	0.329	**0.330**	**0.302**

Table A4: Forecasting Continual Model. Subset Size 5%. Detailed results for Table 2.

Model		Autoformer		PatchTST		TimeMixer		TimesNet		Lag-Llama		Moment		TTM		TimeBlocks		TimeBlocks + Random	
Data Set	*Horizon*	*MAE*	*MSE*	*MAE*	*MSE*	*MAE*	*MSE*	*MAE*	*MSE*	*MAE*	*MSE*	*MAE*	*MSE*	*MAE*	*MSE*	*MAE*	*MSE*	*MAE*	*MSE*
ETTh1	96	0.501	0.549	0.402	0.381	0.397	0.375	0.420	0.399	**0.396**	0.396	0.422	0.414	0.400	0.366	0.397	**0.356**	0.401	0.362
	192	0.499	0.525	0.439	0.432	0.429	0.438	0.448	0.447	**0.422**	0.424	0.450	0.463	0.423	0.391	0.423	**0.389**	0.448	0.397
	336	0.502	0.522	0.472	0.490	0.462	0.507	0.462	0.477	0.470	0.494	0.464	0.493	0.427	0.421	**0.418**	**0.377**	0.455	0.390
	720	0.516	0.519	0.522	0.555	0.487	0.523	0.491	0.506	0.499	0.520	0.477	0.476	0.490	0.538	**0.474**	**0.450**	0.479	0.467
ETTh2	96	0.452	0.426	0.359	0.312	0.345	0.294	0.372	0.325	0.351	0.326	0.497	0.558	0.340	0.282	**0.334**	**0.271**	0.348	0.273
	192	0.488	0.480	0.407	0.385	0.393	0.374	0.418	0.401	0.401	0.394	0.511	0.597	0.379	0.338	**0.377**	**0.330**	0.396	0.332
	336	0.518	0.529	0.429	0.410	0.445	0.438	0.447	0.435	0.441	0.421	0.537	0.629	0.403	0.383	**0.401**	**0.354**	0.417	0.356
	720	0.566	0.645	0.456	0.435	0.451	0.440	0.465	0.457	0.465	0.450	0.559	0.622	0.435	0.441	**0.433**	**0.381**	0.459	0.387
ETTm1	96	0.484	0.512	0.355	0.310	0.359	0.319	0.372	0.334	0.365	0.365	0.553	0.703	0.331	0.359	**0.323**	**0.289**	0.333	0.295
	192	0.526	0.603	0.385	0.360	0.383	0.360	0.413	0.413	0.418	0.430	0.565	0.717	0.371	0.402	**0.366**	**0.329**	0.398	0.332
	336	0.524	0.601	0.399	0.381	0.409	0.395	0.424	0.418	0.421	0.422	0.581	0.726	0.384	0.424	**0.368**	**0.343**	0.383	0.352
	720	0.545	0.632	0.439	0.444	0.439	0.450	0.447	0.460	0.440	0.462	0.603	0.725	0.416	0.575	**0.411**	**0.406**	0.440	0.411
ETTm2	96	0.326	0.243	0.265	0.178	0.259	0.174	0.268	0.187	0.277	0.194	0.546	0.685	0.248	0.174	**0.246**	**0.161**	0.264	0.163
	192	0.347	0.289	0.307	0.243	0.301	0.240	0.307	0.246	0.293	0.239	0.557	0.708	0.288	0.240	**0.287**	**0.215**	0.299	0.219
	336	0.394	0.373	0.345	0.301	0.340	0.297	0.348	0.313	0.328	0.305	0.567	0.715	0.328	0.299	**0.326**	**0.264**	0.339	0.267
	720	0.426	0.432	0.401	0.397	0.398	0.401	0.403	0.410	0.395	0.394	0.578	0.694	0.381	0.407	**0.375**	**0.336**	0.397	0.346
Weather	96	0.331	0.249	0.206	0.160	0.205	0.155	0.219	0.168	0.228	0.168	0.554	0.710	0.196	0.152	**0.192**	**0.144**	0.198	0.145
	192	0.381	0.326	0.243	0.203	0.248	0.201	0.264	0.223	0.272	0.229	0.569	0.729	0.240	0.198	**0.231**	**0.190**	0.235	0.191
	336	0.384	0.346	0.290	0.262	0.280	0.249	0.302	0.279	0.292	0.264	0.583	0.732	0.275	0.250	**0.269**	**0.242**	0.293	0.243
	720	0.422	0.423	0.340	0.336	0.340	0.327	0.353	0.353	0.331	0.331	0.594	0.719	0.338	0.326	**0.325**	**0.305**	0.338	0.321

A.6.2 Effect of the Number Blocks. Even when the linear models are significantly smaller [74], which allows for the construction of very efficient models, as enabled by `TimeBlocks`, we want to assess the efficiency of the router in selecting a specific number of blocks J for a model, and how that affects the performance. Therefore, when building a model, we evaluate the impact of adding a new block and evaluating the model performance up to that point.

The results in Figure A1 show that adding new blocks improves performance. In Figure A1(a), the results are normalized to show the trend among data sets, indicating that when adding new blocks, the normalized error decreases. Further analysis in Figure A1(b) for two data sets reveals that the absolute change is relatively low when new blocks are added. This suggests that a model with few blocks, such as $J = 4$, can achieve similar performance to a larger model, improving the efficiency of the model proposal.



(a) Normalized Error. **(b) Non-normalized Error.**

Figure A1: Model Size Efficiency. Forecasting horizon is 96.

A.6.3 Pre-Training Cost. The pretraining of a large number of blocks in the `BlockBase` is, in principle, comparable to training a single large foundational model, with two significant differences. First, the blocks can be pretrained independently and in parallel, which speeds up training compared to training a single large model. For example, a training step for a six-block model takes less than second, while the same step in the `Lag-Llama` model will take more than a minute, Second, during inference, as shown in Figure 9(a), the `TimeBlocks` approach enables the deployment of lightweight and efficient models, while the large-model approach requires loading the entire large model, leading to considerably higher computational costs.

A.6.4 Coreset Evaluation. To extend the results shown in the last two rows of Table 2, we evaluate `StreamCore` with two coreset methods: `DER++` [5], an experience replay strategy, and `Camel` [35], which implements a buffer. The results for three forecasting data sets in Table A5 show that `StreamCore` maintains a high performance, mainly due to its tight guarantee.

Table A5: Forecasting Continual Model With Coresets (Average MSE).

Coreset	*ETTh1*	*ETTh2*	*Weather*
`DER++`	0.419	0.517	0.346
`Camel`	0.450	0.443	0.312
`StreamCore`	**0.393**	**0.342**	**0.220**

A.6.5 Statistical Breakdown of Block Usage. To evaluate the `Blockbase` usage, a statistical breakdown is shown in Table A6. It considers a sample of five hundred blocks across the four evaluated tasks and related data sets, disaggregated by context length. Thus, each row represents the percentage usage of each particular block type for a given context length.

The main insight from these results is the significant use of the MLP block in all cases. However, it remains relatively difficult to draw firm conclusions regarding block usage across different tasks or domains.

We believe part of the flexibility offered by `TimeBlocks` is its ability to automatically select the most suitable blocks, rather than relying on metadata-based rules. Introducing restrictions based on domain, data set, or task would shift the approach toward such rule-based selection.

Table A6: Statistical Breakdown of Block Usage by Task and Context Length (Block Use Percentage).

Task (Data set) – Context Length	*Block Type*		
	Attention	*LSTM*	*MLP*
Classification (Adiac)			
– 256	0.313	0.000	0.688
– 512	0.667	0.278	0.056
– 1024	0.000	0.000	1.000
– 2048	0.375	0.063	0.563
Forecasting (ETTh1, ETTh2)			
– 256	0.269	0.321	0.410
– 512	0.141	0.218	0.641
– 1024	0.371	0.258	0.371
– 2048	0.238	0.286	0.476
Imputation (ETTm1, Weather)			
– 256	0.000	0.920	0.080
– 512	0.000	0.080	0.920
– 1024	0.800	0.120	0.080
– 2048	0.080	0.080	0.840
Outlier Detection (MSL, SMAP)			
– 256	0.000	0.000	1.000
– 512	0.750	0.000	0.250
– 1024	0.071	0.286	0.643